Workflow patterns

For true end-to-end process orchestration, you must be able to accurately express all the things happening in your business processes which will require simple and advanced workflow patterns. This page describes typical patterns and how you can implement them using Camunda and BPMN.

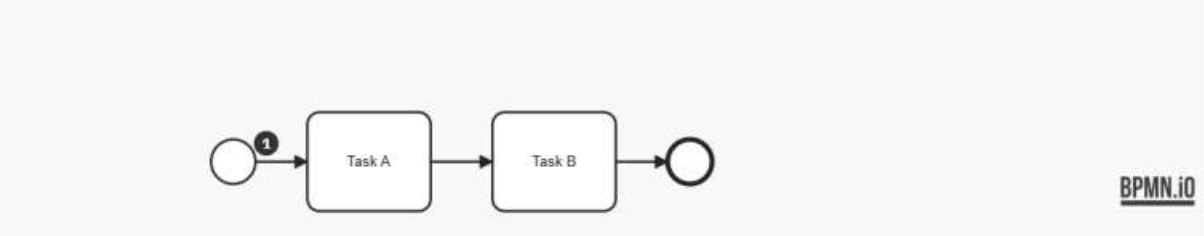
Routing

The most basic workflow patterns are (excuse the play on words) around the basic flow of work.

Sequence

"A task in a process is enabled after the completion of a preceding task in the same process."

This is implemented by a sequence flow connecting two activities:

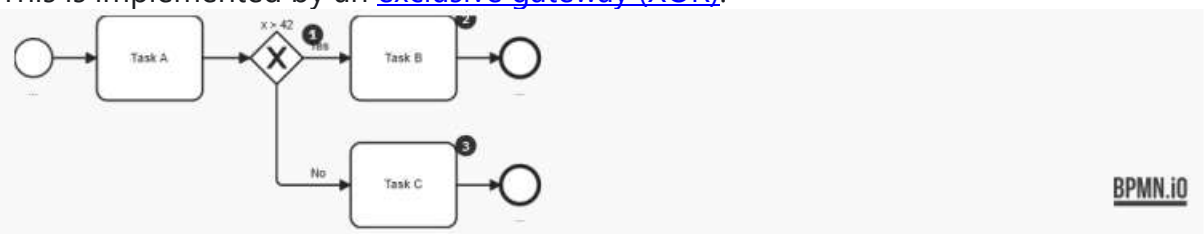


The first sequence flow (1) connects the start event with **Task A**. Then, **Task B** is connected using another sequence flow, meaning that it can only happen if **Task A** was completed. If **Task B** completes, the sequence flow routes to the end event so that the process instance can complete.

Conditions (if/then)

"The thread of control is immediately passed to precisely one of the outgoing branches."

This is implemented by an exclusive gateway (XOR):



All outgoing sequence flows of the XOR gateway (1) have a condition configured, which decides if the process continues in **Task B** (2 if `x>42`) or **Task C** (3if `not(x>42)`).

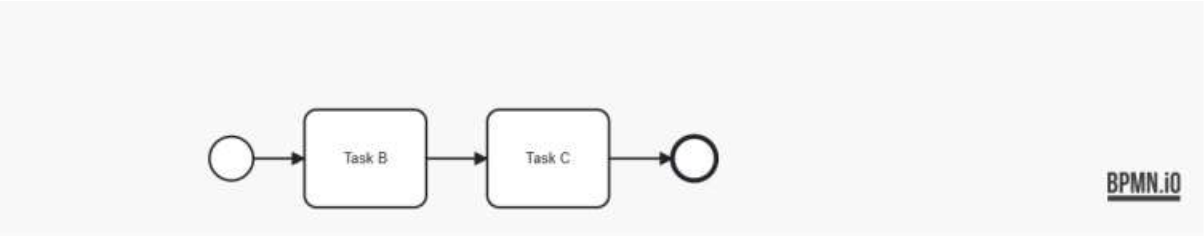
Invoke subworkflows

You need to invoke another process as part of your process.

This is implemented by a call activity:

1) When the call activity is entered, a new process instance of the referenced process is created. Only when the created process instance is completed is the call activity left and the outgoing sequence flow taken.

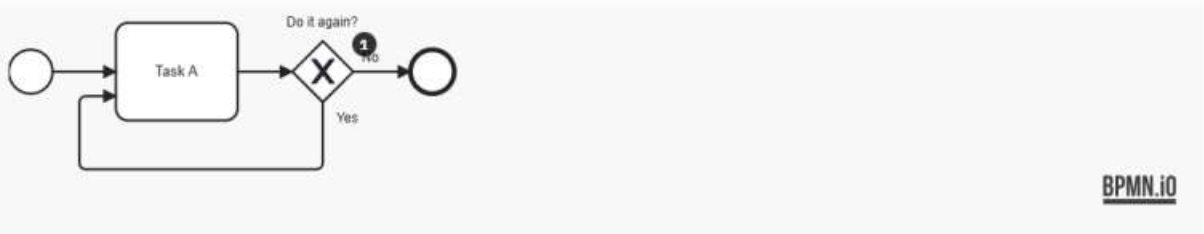
You can reference any other BPMN process, for example:



Loop

"The ability to execute a task or subprocess repeatedly. The loop has either a pre-test or post-test condition associated with it."

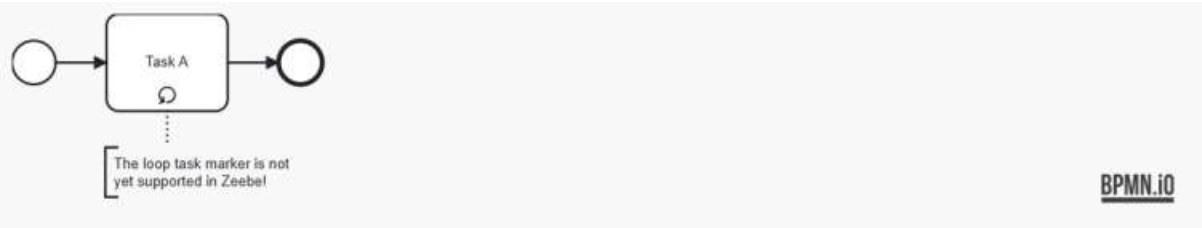
In BPMN, you can simply model a loop:



1) This exclusive gateway contains the expression to decide if to continue or exit the loop. The gateway can be before or after the loop.

There is also a specific loop task marker in BPMN:

```
Task A The loop task marker is not yet supported in Zeebe!
```



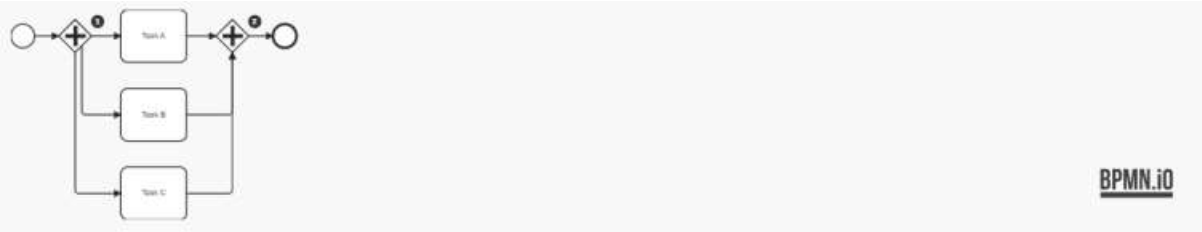
NOTE

The loop task marker event is supported in Camunda 7, but not yet in Camunda 8. It is on the roadmap and will eventually be available in Camunda 8.

Static parallel branches

Imagine you want some tasks known during design time to be carried out in parallel. "The divergence of a branch into two or more parallel branches each of which execute concurrently" plus "the convergence of two or more branches into a single subsequent branch."

In BPMN, this is implemented using [parallel gateways (AND)]:



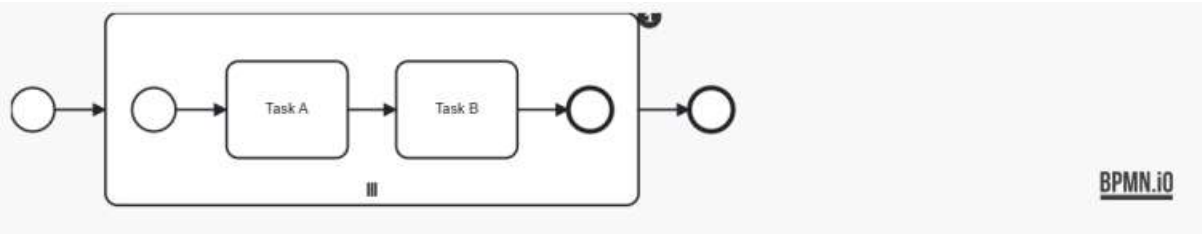
1) This AND-gateway splits the flow into concurrent paths so that Task A, B, and C are executed in parallel.

2 This AND-gateway waits for Task A, B, and C to complete before the flow can move on.

Dynamic parallel branches

You might want to execute some tasks for every element of a list, like the `for each` construct in programming languages. "Multiple instances of a task can be created. The required number of instances may depend on a number of runtime factors, but is known before the task instances must be created. Once initiated, these instances are independent of each other and run concurrently. It is necessary to synchronize the instances at completion before any subsequent tasks can be triggered."

In BPMN, this is implemented using [multiple instance activities]:

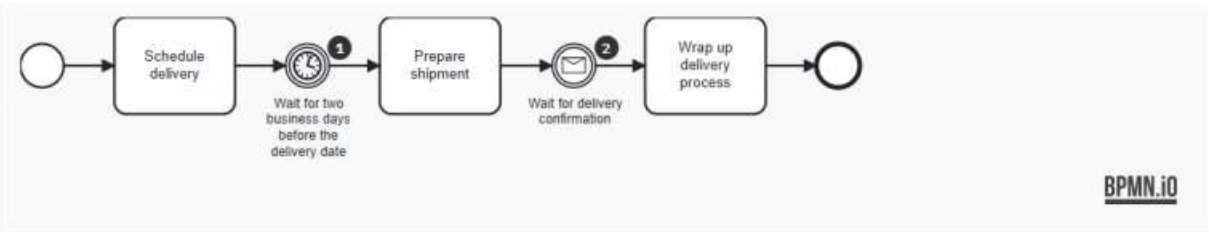


1) The parallel multiple instance marker defines that this subprocess is executed multiple times - once for each element of a given collection (like a `for each` loop in a programming language).

Wait

A typical situation is that a process needs to wait for some event to happen, e.g. some time to pass or some external message to arrive.

In BPMN, this is implemented using events (or receive tasks):



1 The timer event causes the process to wait, in this case until a specific point in time is due or some duration has elapsed.

2 The process will wait for a message to arrive. The message is an external trigger provided by API and can technically be anything, from a callback (e.g. via REST), over real messaging (like AMQP), or to notifications within your system.

Reacting to events

The waiting mentioned above is a special case where you react to events while not doing anything else. Oftentimes, you want to react to events even if the process is doing something else at the moment. This is described in this section.

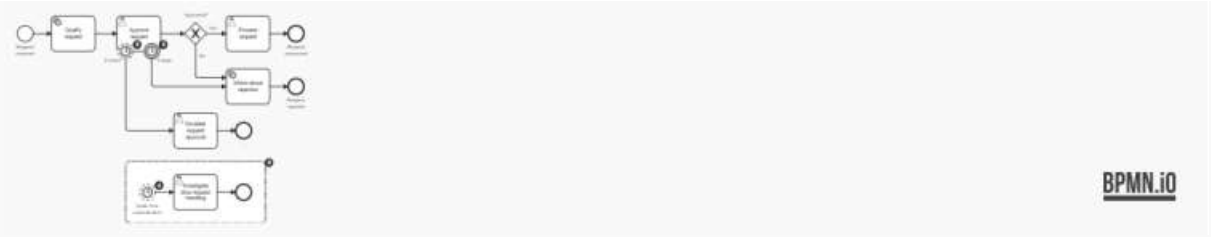
Typical examples are customer cancelation requests coming in for running order fulfillment processes, or timeouts if parts of the process take too long.

Time based

You want to react if a certain point in time is due or a specific time duration has passed.

In BPMN, you can leverage boundary events or event subprocesses.

Those events can be interrupting or non-interrupting, meaning you will either interrupt the current activity, or start something in parallel.



1 This timer is non-interrupting (dashed line), so the **Escalate request approval** task is started in parallel, additionally to the **Approve request** task. The idea is that the escalation task might make a manager double-checking the original task does not slip. Non-interrupting events can also be recurring, so you could also escalate "every two hours".

2 This timer is interrupting (solid line). Once it fires, the **Approve request** task is canceled and the process continues on the alternative path, in this case to automatically reject the request. Note that both timers so far can only happen if the task **Approve request** is active.

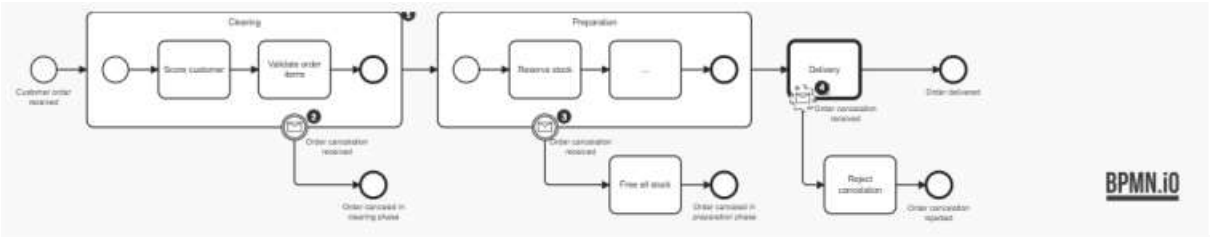
3 This is an event subprocess (dotted line). This can be activated from everywhere in the current scope. In this example, the scope is the whole process.

4 So if the process is not completed within the defined SLA, the timer fires and the event subprocess is started. As the timer is non-interrupting (dashed line again), it does not intervene with the typical flow of operations, but starts something additionally in parallel.

External messages/events

You might also want to react to certain incoming messages or events in an existing process. A good example is a customer canceling the current order fulfillment process. This might be possible only in a certain process phase and lead to different actions.

As with timers, you can leverage boundary events or event subprocesses.



Assume that an order cancelation message comes in for the current process instance using [message correlation](#).

1 Subprocesses can be easily used to define phases of a process, as the cancelation is treated differently depending on the current process phase.

2 For example, a cancelation during the clearing phase has no consequences and can simply be executed.

3 But when the process is already in the preparation phase it might need to clean up certain things properly.

4 During delivery, it does not even allow cancelations anymore. This is also why this event is non-interrupting (dashed line), so we keep doing **Delivery**.

Correlation mechanisms

Mapping external messages to an existing process instance is called [message correlation](#). This is a crucial functionality to ensure you can communicate with process instances from the outside.

There are two main problems to solve:

1. How to find the right process instance? In Camunda, this is solved by a `message name` and a `correlation key` (e.g. `orderCanceled` and `order-42`).
2. How to persist messages if a process instance is not yet ready to receive that message yet? In Camunda, this is solved by having an internal message store and a `time to live` attached to messages.

Events from subprocesses

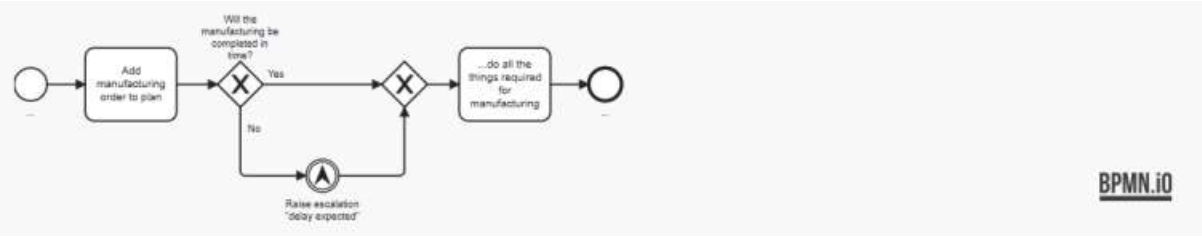
Sometimes, a subprocess needs to communicate with its parent process without ending the subprocess yet. BPMN allows this by an [escalation event](#).

NOTE

The escalation event is supported in Camunda 7, but not yet in Camunda 8. It is on the roadmap and will eventually be available in Camunda 8.

1 An escalation event can be thrown from any of the called subprocesses and is picked up by its parent to start something in parallel, as this is a non-interrupting event (dashed line).

The subprocess can raise the escalation any time:

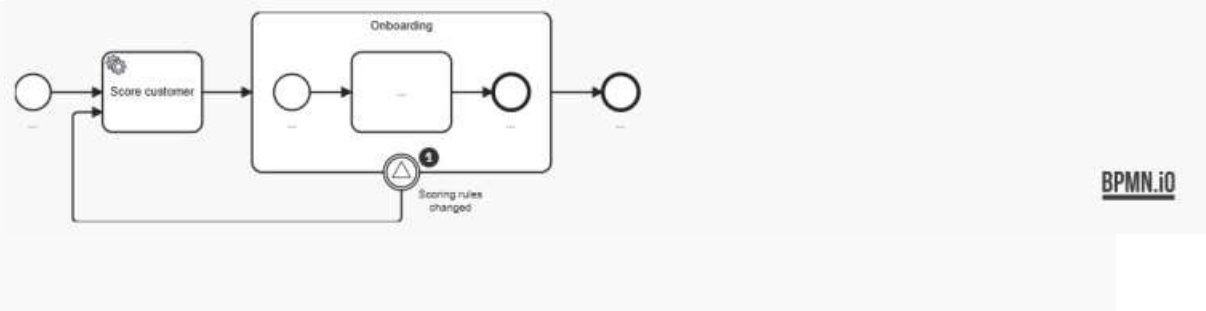


Broadcasts and engine-wide events

While messages are always targeted at one specific process instance, you might also want to inform many processes about an event at once. For example, you might regularly adjust certain customer scoring rules that always should be taken into account immediately. This can be implemented using the signal event.

NOTE

The escalation event is supported in Camunda 7, but not yet in Camunda 8. It is on the roadmap and will eventually be available in Camunda 8.



1 The signal event is caught and in this case interrupts the onboarding to go back to score the customer again.